A Lyapunov Drift-Plus-Penalty Method Tailored for Reinforcement Learning with Queue Stability

Wenhan Xu, Graduate Student Member, IEEE, Jiashuo Jiang, Member, IEEE, Lei Deng, Member, IEEE, Yuan Wu, Senior Member, IEEE, and Danny H.K. Tsang, Life Fellow, IEEE

arXiv:2506.04291v2 [cs.LG] 13 Aug 2026

Abstract—With the proliferation of Internet of Things (IoT) devices, the demand for addressing complex optimization challenges has intensified. The Lyapunov Drift-Plus-Penalty algorithm is a widely adopted approach for ensuring queue stability, and several researchers has preliminarily explored its integration with reinforcement learning (RL). In this paper, we investigate the adaptation of the Lyapunov Drift-Plus-Penalty algorithm for RL applications, deriving an effective method for combining Lyapunov Drift-Plus-Penalty with RL under a set of common and reasonable conditions through rigorous theoretical analysis. Unlike existing approaches that directly merge the two frameworks, our proposed algorithm, termed Lyapunov drift-plus-penalty method tailored for RL with queue stability (LDPTRLQ) algorithm, offers theoretical superiority by effectively balancing the greedy optimization of Lyapunov Drift-Plus-Penalty with the long-term perspective of RL. Simulation results for multiple problems demonstrate that LDPTRLQ outperforms the baseline methods using the Lyapunov drift-plus-penalty method and RL, corroborating the validity of our theoretical derivations. The results also demonstrate that our proposed LDPTRLQ outperforms other benchmarks in terms of compatibility and stability.

Index Terms—Online Optimization, Reinforcement Learning, Lyapunov Drift-Plus-Penalty

I. Introduction

The Lyapunov drift-plus-penalty framework has emerged as a widely adopted technique across diverse paradigms of communication systems, including backpressure routing [1], Internet of Things (IoT) devices, adaptive video streaming [2], and edge computing [3]. This method excels in stabilizing queues—both physical and virtual—within queueing networks and other stochastic systems while simultaneously optimizing time-averaged performance objectives, such as minimizing a penalty function [4]. Its major strengths are implementation efficiency and simplicity. Unlike traditional approaches, the Lyapunov drift-plus-penalty method requires no prior knowledge of underlying statistical distributions (e.g., probability density functions of random channel conditions). Instead, it relies solely on greedy optimization of arbitrary utility functions at each time step. This lightweight and scalable design eliminates the need for complex probabilistic modeling, rendering the technique highly adaptable and broadly applicable [5].

Although the Lyapunov drift-plus-penalty method offers simplicity in implementation [6], its effectiveness is often constrained by the complexity of penalty functions inherent in many optimization problems. These penalty functions, typically dictated by the optimization objectives of specific scenarios, are frequently complex and non-convex. While they remain computable, sometimes they do not inherently possess greedy properties, limiting the method's applicability in such cases [7]. Consequently, researchers have explored alternative approaches to address these challenges.

Recent advancements in data-driven reinforcement learning (RL) present a promising alternative for tackling online optimization problems in queueing systems [8]. This method autonomously learns from past experiences without requiring manually labeled training data, making it particularly well-suited for queueing problems that demand real-time, online implementation. Recent research has predominantly explored direct combinations of the Lyapunov drift-plus-penalty method and RL algorithms to tackle optimization challenges in stochastic systems [9]. The Lyapunov drift-plus-penalty method operates as a greedy algorithm, optimizing decisions based on immediate outcomes, whereas RL employs non-greedy exploration to maximize long-term rewards. As a result, devising a method that effectively unifies these two frameworks remains a complex challenge.

A. Related Work

The Lyapunov drift-plus-penalty method, first introduced by Neely in 2010 [10] to stabilize queues while optimizing specific objectives, has since found widespread application in queueing-related optimization problems, including backpressure routing [1], IoT resource allocation [11], and edge computing [12]. This approach transforms long-term queue stability goals into per-slot minimization of a drift-plus-penalty term, employing a greedy

Wenhan Xu is with the Internet of Things Thrust, Hong Kong University of Science and Technology (Guangzhou), Guangzhou, Guangdong 511400, China (e-mail: wxube@connect.ust.hk).

Jiashuo Jiang is with Department of Industrial Engineering and Decision Analytics, Hong Kong University of Science and Technology, Hong Kong 99907, China (e-mail: jsjiang@ust.hk).

Lei Deng is with the Internet of Things Thrust, Hong Kong University of Science and Technology (Guangzhou), Guangzhou, Guangdong 511400, China (e-mail: leideng@hkust-gz.edu.cn).

Yuan Wu is with The State Key Lab of Internet of Things for Smart City, and also with the Department of Electronic and Communication Engineering, University of Macau, Macao, China (e-mail: yuanwu@um.edu.mo).

Danny Hin-Kwok Tsang is with the Internet of Things Thrust, Hong Kong University of Science and Technology (Guangzhou), Guangzhou, Guangdong 511400, China, and also with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Clear Water Bay, Hong Kong SAR, China (e-mail: eetsang@ust.hk).

optimization strategy that requires no prior statistical knowledge of the system (e.g., stochastic probability density functions). Bracciale et al. [13] developed a system based on this method, enforcing not only average queue stability but also strict per-slot maximum queue (or delay) capacity constraints. Similarly, [14] proposed an online Lyapunov-based approach, investigating centralized and distributed schemes for joint task offloading and resource allocation in energy-constrained mobile edge computing (MEC) networks. Furthermore, Hu et al. [15] implemented the Lyapunov drift-plus-penalty framework to obtain a dynamic resource allocation policy in a stochastic network optimization problem.

Concurrently, model-free RL algorithms, typically leveraging deep neural networks, learn effective policies through iterative trial-and-error interactions, rendering them well-suited for a variety of online optimization problems [16]. In contrast to the Lyapunov drift-plus-penalty technique, which prioritizes per-slot optimization, RL focuses on maximizing long-term rewards, offering a fundamentally different approach to system dynamics. Implementing an RL framework requires only a standardized specification of actions, states, rewards, and state transitions [17]. The significance of RL in constrained markov decision processes (CMDP) contexts has garnered substantial attention in recent years, particularly for systems requiring safety or resource guarantees [18]. A variety of methods for tackling CMDP have been developed to balance objective maximization with constraint satisfaction [19], [20]. This versatility has led to its widespread adoption across diverse real-world applications, including large language models [21], image recognition [22], and perfect-information games [23].

Recent efforts have sought to integrate the Lyapunov drift-plus-penalty method with RL to harness their complementary strengths [24]–[28]. For instance, researchers in [24] incorporated Lyapunov optimization objectives directly into RL reward functions to address edge computing challenges in Digital Twin Networks. Similarly, Bi et al. [25] applied a hybrid framework to mobile edge computing, using Lyapunov terms to guide RL policy updates. In [26], a channel selection framework was proposed, combining machine learning, Lyapunov optimization, and matching theory to support industrial IoT implementations. Zhou et al. [27] proposed a predictable radio resource scheduling scheme leveraging Lyapunov-guided RL with proximal policy optimization. While the Lyapunov drift-plus-penalty framework provides a principled approach for long-term time-averaged optimization, its direct integration with RL poses practical challenges, where the greedy nature of the Lyapunov drift-plus-penalty method conflicts with RL's focus on maximizing long-term rewards.

B. Motivation and Contribution

The rapid proliferation of portable IoT devices and applications has significantly increased the demand for optimization frameworks that ensure queue stability while addressing increasingly complex objectives. These scenarios, spanning domains such as backpressure routing and edge computing, often involve dynamic, non-convex optimization targets tied to practical performance metrics. Concurrently, advancements in artificial intelligence have prompted researchers to explore superior algorithmic alternatives, even in well-established fields like backpressure routing. The Lyapunov drift-plus-penalty method has emerged as a popular approach for optimizing application-specific objectives under queue stability constraints, leveraging its greedy, per-slot minimization strategy [10], [13], [14], [29], [30]. However, the dynamic, non-convex, and intricate nature of modern optimization goals has driven efforts to integrate this method with data-driven, model-free RL algorithms [24]–[28]. While such hybrid approaches have demonstrated performance gains in specific contexts, there exists a critical mismatch, namely, the greedy optimization inherent to the Lyapunov framework conflicts with RL's focus on maximizing long-term rewards, often leading to suboptimal or unstable outcomes. Our prior research investigated joint energy consumption optimization and queue stability in intelligent reflecting surface-assisted fog computing [31], [32], highlighting the potential of tailored approaches in complex scenarios.

Motivated by the insights from the aforementioned literature review, we propose a novel reformulation of the Lyapunov drift-plus-penalty method, adapting it to align with RL's non-greedy paradigm. This paper presents several key contributions to address the integration of the Lyapunov drift-plus-penalty method with RL for queueing system optimizations.

- We conduct a theoretical analysis of the Lyapunov drift-plus-penalty method. Based on the practical significance of optimization objectives, we propose a set of basic theorems that such objectives should exhibit. Leveraging these theorems, we propose a Lyapunov drift-plus-penalty method tailored for RL with queue stability (LDPTRLQ) algorithm, ensuring compatibility with its non-greedy dynamics while preserving queue stability.
- We propose a dynamic Lyapunov weight selection algorithm to address scenarios where the Lyapunov weight is not predetermined in practical settings. Additionally, we introduce a saturated queue bailout mechanism that enables the LDPTRLQ algorithm to handle cases where the arrival rate of certain queues approaches the service rate limit.
- We validate the performance of the proposed RL-adapted Lyapunov drift-plus-penalty algorithm in a mobile edge computing problem, and compare it against the widely adopted original Lyapunov drift-plus-penalty-assisted RL approach. Numerical results demonstrate that our algorithm achieves better optimization performance and faster convergence. We also validate the performance of the proposed RL-adapted Lyapunov drift-plus-penalty algorithm in a routing problem. The results demonstrate that

TABLE I
Parameters notation

Parameters	Notation
n	n^{th} queue
N	Number of queues
Q	Length of queue
$\mathbb{E}\{x\}$	Expectation of x
$\mathbb{R}$	Real number field
d^{arr}	Arrival of the queue
d^{dep}	Departure of the queue
t	t^{th} time slot
ΔT	Length of time slot
$\mathcal{C}$	Lyapunov drift-plus-penalty function
V	Weight between Lyapunov drift and penalty
$\overline{p}$	Time-averaged Lyapunov penalty
$p(t)$	Lyapunov penalty at time slot t
$\boldsymbol{a}$	Action of RL algorithm
$\boldsymbol{s}$	State of RL algorithm
$\mathcal{A}$	Set of feasible actions
$\mathcal{S}$	Set of feasible states
$\mathcal{R}$	Reward of RL
λ	Task arriving rate
K	Number of mobile users

LDPTRLQ achieves performance comparable to the established backpressure routing algorithm while surpassing other hybrid approaches that integrate Lyapunov Drift-Plus-Penalty with RL. Across a variety of queueing optimization problems, our proposed LDPTRLQ algorithm demonstrates superior performance. These results underscore the algorithm's robust generality and scalability, highlighting its potential for broad applicability in diverse optimization scenarios.

C. Notations

Scalars and vectors are denoted by letters and bold letters, respectively. $|x|$ is the absolute value of the scalar x. $\|\boldsymbol{x}\|$ is the L_2-norm of vector $\boldsymbol{x}$. $\mathbf{X}^T$, $\mathbf{X}^H$, and $|\mathbf{X}|$ are the transpose, conjugate transpose, and determinant norm of a matrix $\mathbf{X}$, respectively. The primary notations in this paper are summarized in TABLE I.

II. System Model and Background

In this section, we present the theoretical foundations of the Lyapunov drift-plus-penalty method applied to queueing systems. This framework provides a structured approach to stabilize queues while optimizing performance objectives, laying the groundwork for our subsequent analysis and algorithm design.

A. Optimization Problem with Queueing Systems

Consider a system comprising N distinct queues. The queue lengths over time are represented by the following vector:

$$\boldsymbol{Q}(t) = [Q_1(t), Q_2(t), \cdots, Q_n(t), \cdots, Q_N(t)], \quad (1)$$

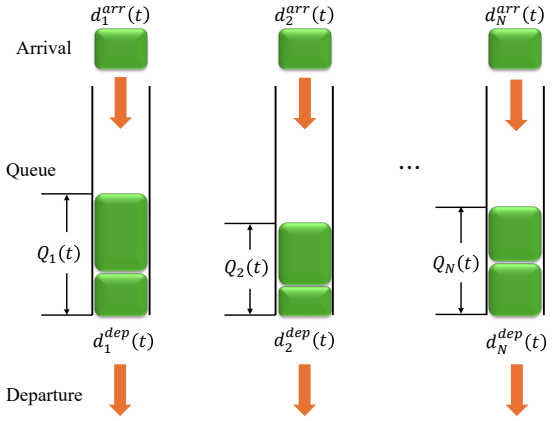


Fig. 1. System model of N queues.

where $Q_n(t) \geq 0$ denotes the length of the nth queue at time slot $t \in \{0, 1, 2, 3, \cdots\}$. To characterize the evolution of queue lengths, we denote the arrival process for the nth queue at time slot t as $d_n^{\text{arr}}(t)$, and the departure process as $d_n^{\text{dep}}(t)$. The evolution of the queue lengths can be described by the following queueing model:

$$Q_n(t+1) = \max\{Q_n(t) - d_n^{\text{dep}}(t), 0\} + d_n^{\text{arr}}(t). \quad (2)$$

Definition 1. [6] A discrete time queue Q_n is mean rate stable if

$$\lim_{t\to\infty} \frac{\mathbb{E}\{Q_n(t)\}}{t} = 0, \quad (3)$$

where $\mathbb{E}\{Q_n(t)\}$ is the expectation of $Q_n(t)$.

After establishing the queue dynamics, we now formulate the optimization problem based on the Lyapunov drift-plus-penalty framework [13]. The objective is to find a policy to minimize the average penalty with the mean rate stable queue:

$$\begin{aligned} &\mathcal{P}1: \min \overline{p} \\ &\text{s.t. } Q_n \text{ is mean rate stable}, \forall n, \\ &\alpha(t) \in \mathcal{A}(t), \end{aligned} \quad (4)$$

where $\alpha(t)$ represents the action at time slot t that the optimizer needs to decide, selected from the set of feasible actions $\mathcal{A}(t)$, and $\overline{p}$ is the time-averaged penalty excluding the queue stability constraint:

$$\overline{p} = \lim_{\tau\to\infty} \frac{\sum_{t=0}^{\tau-1} \mathbb{E}\{p(t)\}}{\tau}, \quad (5)$$

where $p(t)$ denotes the penalty incurred at time slot t. Different specific problems have different $p(t)$ expressions. In practical applications, $p(t)$ typically represents the time-averaged cost-related metrics such as the energy consumption.

B. Lyapunov Drift-Plus-Penalty Framework

After establishing the basic optimization problem, we now turn to the Lyapunov drift-plus-penalty framework.

The key here is the introduction of a Lyapunov function based on the queue vector $Q(t)$, defined as follows

$$LY(\boldsymbol{Q}(t)) \triangleq \frac{1}{2}\sum_{n=1}^{N} Q_n^2(t), \tag{6}$$

where $LY(\boldsymbol{Q}(t))$ is the Lyapunov function of $Q(t)$. Accordingly, the Lyapunov drift at time slot t is defined as the difference between the Lyapunov function values of consecutive time slots, expressed as

$$\begin{aligned}\Delta\boldsymbol{Q}(t) \triangleq& \mathbb{E}\{LY(\boldsymbol{Q}(t+1)) - LY(\boldsymbol{Q}(t))|\boldsymbol{Q}(t)\}\\ =& \mathbb{E}\{\frac{1}{2}\sum_{n=1}^{N} Q_n^2(t+1) - \frac{1}{2}\sum_{n=1}^{N} Q_n^2(t)|\boldsymbol{Q}(t)\}\\ =& \mathbb{E}\{\frac{1}{2}\sum_{n=1}^{N}(Q_n^2(t+1) - Q_n^2(t))|\boldsymbol{Q}(t)\}.\end{aligned} \tag{7}$$

Therefore, the Lyapunov drift-plus-penalty function at time slot t is

$$\begin{aligned}\mathcal{C}(t) =& \underbrace{\Delta\boldsymbol{Q}(t)}_{\text{Drift}} + \underbrace{V\mathbb{E}\{p(t)|\boldsymbol{Q}(t)\}}_{\text{Penalty}}\\ =& \underbrace{\mathbb{E}\{\frac{1}{2}\sum_{n=1}^{N}(Q_n^2(t+1) - Q_n^2(t))|\boldsymbol{Q}(t)\}}_{\text{Drift}}\\ &+ \underbrace{V\mathbb{E}\{p(t)|\boldsymbol{Q}(t)\}}_{\text{Penalty}},\end{aligned} \tag{8}$$

where V is the weight between Lyapunov drift and penalty. By manipulating the controllable components of the optimization environment, the Lyapunov drift-plus-penalty method performs the following optimization problem (i.e., P2) at each time slot:

$$\begin{aligned}&\mathcal{P}2: \min_{\alpha(t)} \mathcal{C}(t)\\ &\text{s.t. } \alpha(t) \in \mathcal{A}(t),\end{aligned} \tag{9}$$

where $\alpha(t)$ represents the action at time slot t selected from the set of feasible actions $\mathcal{A}(t)$, and the Lyapunov drift-plus-penalty function $\mathcal{C}(t)$ is a function of the action $\alpha(t)$. Since the optimization process is confined to a single time slot and $Q(t)$ is a known quantity during per-slot optimization, the expectation terms in the Lyapunov drift-plus-penalty function $\mathcal{C}(t)$ can be replaced with their current values, yielding

$$\begin{aligned}&\mathcal{P}3: \min_{\alpha(t)} \frac{1}{2}\sum_{n=1}^{N}(Q_n^2(t+1) - Q_n^2(t)) + Vp(t)\\ &\text{s.t. } \alpha(t) \in \mathcal{A}(t).\end{aligned} \tag{10}$$

In several related works such as [13], [25], the arrival length $d^{arr}(t)$ and departure length $d^{dep}(t)$ of queues within a single time slot are treated as bounded quantities and significantly smaller than the queue length $Q(t)$. Consequently, these existing works employ simplification and approximation techniques to transform problem $\mathcal{P}3$ into problem $\mathcal{P}4$ as follows

$$\begin{aligned}&\mathcal{P}4: \min_{\alpha(t)} \sum_{n=1}^{N} Q_n(t)(d_n^{\text{arr}}(t) - d_n^{\text{dep}}(t)) + Vp(t)\\ &\text{s.t. } \alpha(t) \in \mathcal{A}(t).\end{aligned} \tag{11}$$

III. Lyapunov Drift-Plus-Penalty Method for RL

Typically, the greedy optimization problems $\mathcal{P}3$ and $\mathcal{P}4$ can be solved directly. However, when $p(t)$ exhibits a complex, non-convex form, solving $\mathcal{P}3$ and $\mathcal{P}4$ may become NP-hard, rendering them computationally intractable. Furthermore, in practical optimization scenarios, the greedy minimization of the penalty $p(t)$ often fails to reduce its long-term time-averaged value, indicating weak greedy properties of the penalty term. In such cases, even if $\mathcal{P}3$ and $\mathcal{P}4$ are solvable, the resulting solutions yield suboptimal performance. This limitation has motivated researchers to explore model-free optimization techniques, such as deep learning. In this section, we aim to derive a theoretically sound adaptation of the Lyapunov drift-plus-penalty method for RL applications, addressing these challenges through rigorous analysis.

A. Theoretical Derivation of the Proposed LDPTRLQ Method

1) Derivation of the form of optimization problem: The following theorem builds on the widespread application of the Lyapunov drift-plus-penalty method to queue stability problems. This method's established efficacy in such contexts provides the foundation for our subsequent adaptation to RL frameworks.

Theorem 1. The Lyapunov drift-plus-penalty method provides an effective framework for addressing queue stability problems. Specifically, its effectiveness means that solving the optimization problem $\mathcal{P}3$ at each time slot can achieve the desired optimization objective while ensuring queue stability.

Theorem 1 is proven in section 3.2 of [10], and the successful applications of the Lyapunov drift-plus-penalty method in [13], [14], [29], [30] further validates Theorem 1. In RL methodologies, the optimization objective is typically transformed directly into the reward function, with the action space and state space derived from the original optimization problem's decision variables and feasible domain, respectively. A key advantage of RL is that it requires only the ability to compute the reward, which is equivalent to the optimization objective, rendering approximation or simplification of the reward unnecessary. Such simplifications, as applied in transforming $\mathcal{P}3$ to $\mathcal{P}4$, can even degrade RL efficiency. Consequently, we focus solely on the original problem $\mathcal{P}3$. Based on Theorem 1, we deduce that the optimization problem we seek is equivalent to $\mathcal{P}3$. In terms of the equivalent optimization problem, we have Theorem 2.

Theorem 2. All problems equivalent to a given optimization problem can be characterized by applying a monotonically increasing function to its objective function:

$$\min_{\alpha\in\mathcal{A}} f(\alpha) \iff \min_{\alpha\in\mathcal{A}} \phi(f(\alpha)) \tag{12}$$

where ϕ is a monotonically increasing function with a single independent variable.

Proof. We have an optimization problem $\mathcal{P}3a$ defined as

$$\mathcal{P}3a : \min_{\alpha\in\mathcal{A}} f(\alpha), \tag{13}$$

and another optimization problem $\mathcal{P}3b$ defined as

$$\mathcal{P}3b : \min_{\alpha\in\mathcal{A}} g(\alpha). \tag{14}$$

where $g(\alpha) = \phi(f(\alpha))$, and $\phi : \mathbb{R} \to \mathbb{R}$ is a monotonically increasing function. If the optimal solutions of $\mathcal{P}3a$ and $\mathcal{P}3b$ are identical, the equivalence can be proven. Let $\alpha^* \in \mathcal{A}$ be an optimal solution to $\mathcal{P}$, so that $f(\alpha^*) \leq f(\alpha), \forall\alpha \in \mathcal{A}$. Since ϕ is monotonically increasing, applying ϕ to both sides of the inequality preserves the order: $\phi(f(\alpha^*)) \leq \phi(f(\alpha)), \forall\alpha \in \mathcal{A}$. Thus, there exists

$$g(\alpha^*) = \phi(f(\alpha^*)) \leq \phi(f(\alpha)) = g(\alpha), \quad \forall\alpha \in \mathcal{A}. \tag{15}$$

This implies that α^* is also an optimal solution to $\mathcal{Q}$. Conversely, suppose $\beta^* \in \mathcal{X}$ is an optimal solution to $\mathcal{P}3b$, so that

$$g(\beta^*) = \phi(f(\beta^*)) \leq \phi(f(\alpha)) = g(\alpha), \quad \forall\alpha \in \mathcal{A}. \tag{16}$$

Since ϕ is monotonically increasing, it is injective (one-to-one). Thus, $\phi(f(\beta^*)) \leq \phi(f(\alpha))$ implies $f(\beta^*) \leq f(\alpha)$ for all $\alpha \in \mathcal{A}$, because if $f(\beta^*) > f(\alpha)$, then $\phi(f(\beta^*)) > \phi(f(\alpha))$, contradicting the optimality of β^* in $\mathcal{P}3b$. Hence, β^* is also an optimal solution to $\mathcal{P}3a$. Therefore, the optimal solution sets of $\mathcal{P}3a$ and $\mathcal{P}3b$ are identical, proving that $\mathcal{P}3b$ is equivalent to $\mathcal{P}3a$.

□

Indeed, all optimization problems equivalent to $\mathcal{P}3$ with different objectives can be expressed in the following form

$$\begin{aligned} &\mathcal{P}5 : \min_{\alpha(t)} f_{1,t}\Big(\frac{1}{2}\sum_{n=1}^{N}(Q_n^2(t+1) - Q_n^2(t)) + Vp(t)\Big) \\ &\text{s.t. } \alpha(t) \in \mathcal{A}(t), \end{aligned} \tag{17}$$

where $f_{1,t}, \forall t$ is a monotonically increasing function with a single independent variable. Since $f_{1,t}(x), \forall t$ achieves its minimum value when x attains the minimum within its domain, this transformation preserves the solution to the minimization problem.

2) Derivation of undetermined term in the optimization problem: The reward function for the RL algorithm we seek for $\mathcal{P}5$ can be formulated as follows

$$\mathcal{R}(t) = -f_{1,t}\Big(\frac{1}{2}\sum_{n=1}^{N}(Q_n^2(t+1) - Q_n^2(t)) + Vp(t)\Big). \tag{18}$$

According to Theorem 2 and its proof, we have Lemma 1.

Lemma 1. Eq. (18) represents the set of all reward functions that satisfy the conditions of Theorem 1.

We will gradually narrow down the scope of the set in Lemma 1 based on some properties that RL should satisfy when dealing with this problem. We also establish the following Theorem 3.

Theorem 3. Existence theorem: Since Eq. (18) represents the set of all reward functions, there exists at least one form of Eq. (18) that is an optimal and static reward function.

The premise for Theorem 3 to hold is that the Lyapunov drift plus penalty algorithm can be used in RL. Many studies, e.g., [13], [14], [29], [30], have confirmed this. Therefore, we regard Theorem 3 as an axiom. As our goal is to devise a general method applicable to arbitrary environments, we can first focus on a special case where the queue length is independent of the action α. By Theorem 1, in this case, the greedy policy that minimizes the per-slot objective in each time slot is optimal with respect to the long-run time-average objective. We have the original problem $\mathcal{P}6$ and the target problem $\mathcal{P}7$ suitable for RL:

$$\begin{aligned} &\mathcal{P}6 : \min_{\alpha(t)} Vp(t) + C(t) \\ &\text{s.t. } \alpha(t) \in \mathcal{A}(t), \end{aligned} \tag{19}$$

and

$$\begin{aligned} &\mathcal{P}7 : \min_{\alpha(t)} f_{1,t}(Vp(t) + C(t)) \\ &\text{s.t. } \alpha(t) \in \mathcal{A}(t), \end{aligned} \tag{20}$$

where $C(t) = \frac{1}{2}\sum_{n=1}^{N}(Q_n^2(t+1) - Q_n^2(t))$ is independent of the action α. Since $C(t)$ is a constant independent of the optimization variables, and by Theorem 2, it can be safely removed from the objective function without affecting the optimal solution. Consequently, the term $C(t)$ is omitted, yielding the simplified problems $\mathcal{P}6a$ and $\mathcal{P}7a$ as follows:

$$\begin{aligned} &\mathcal{P}6a : \min_{\alpha(t)} Vp(t) \\ &\text{s.t. } \alpha(t) \in \mathcal{A}(t), \end{aligned} \tag{21}$$

and

$$\begin{aligned} &\mathcal{P}7a : \min_{\alpha(t)} f_{1,t}(Vp(t)) \\ &\text{s.t. } \alpha(t) \in \mathcal{A}(t). \end{aligned} \tag{22}$$

The greedy minimization of the original objective achieves the minimization of its long-term time-averaged value. To achieve the long-term time-averaged value of $\mathcal{P}6a$, we need to solve

$$\begin{aligned} &\mathcal{P}8 : \min_{\alpha} \lim_{\tau\to\infty} \frac{\sum_{t=0}^{\tau-1} Vp(t)}{\tau} \\ &\text{s.t. } \alpha \in \mathcal{A}. \end{aligned} \tag{23}$$

According to Theorem 2, $\mathcal{P}8$ is equivalent to

$$\begin{aligned} &\mathcal{P}9 : \min_{\alpha} \lim_{\tau\to\infty} \frac{f_{1,t'}(\sum_{t=0}^{\tau-1} Vp(t))}{\tau}, \forall t', \\ &\text{s.t. } \alpha \in \mathcal{A}, \end{aligned} \tag{24}$$

where t' can be any integer. The RL algorithm to solve $\mathcal{P}7a$ will look for the minimum long-term time-averaged value

$$\begin{aligned}&\mathcal{P}10: \min_{\alpha} \lim_{\tau\to\infty} \frac{\sum_{t=0}^{\tau-1} f_{1,t}(Vp(t))}{\tau}\\ &\text{s.t. } \alpha \in \mathcal{A}.\end{aligned} \tag{25}$$

According to Theorem 2, $\mathcal{P}10$ is equivalent to

$$\begin{aligned}&\mathcal{P}11: \min_{\alpha} \lim_{\tau\to\infty} \frac{f_2(\sum_{t=0}^{\tau-1} f_{1,t}(Vp(t)))}{\tau}\\ &\text{s.t. } \alpha \in \mathcal{A},\end{aligned} \tag{26}$$

where f_2 is a monotonically increasing function with a single independent variable. According to the assumption of this special case when the queue length is independent of the action α, $\mathcal{P}11$ is equivalent to $\mathcal{P}9$. Therefore,

$$f_{1,t'}(\sum_{t=0}^{\tau-1} Vp(t)) = f_2(\sum_{t=0}^{\tau-1} f_{1,t}(Vp(t))), \forall t'. \tag{27}$$

Eq. (27) holds true for any value of $\{p(t), \tau, V\}$. Let $\tau = 1, t' = 0$, we can get

$$f_{1,0}(Vp(0)) = f_2(f_{1,0}(Vp(0))). \tag{28}$$

Let $x = f_{1,0}(Vp(0))$ and we can get $f_2(x) = x$. Therefore,

$$f_{1,t'}(\sum_{t=0}^{\tau-1} Vp(t)) = \sum_{t=0}^{\tau-1} f_{1,t}(Vp(t)), \forall t'. \tag{29}$$

Lemma 2. Eq. (29) holds true for any value of $\{p(t), \tau, V, t'\}$ if and only if $f_{1,t}(x) = b_1(x + b_{0,t})$ with $b_1 > 0$.

Proof. To achieve the additive property of performance metrics across the time domain, the function f must be linear. If f is nonlinear, then the function value obtained through time averaging will not be equal to the function value evaluated at the average state (as indicated by Jensen's inequality). □

We do not differentiate whether the Lyapunov drift-plus-penalty function is scaled to $\mathcal{C}b_1$ or $\mathcal{C}/b_1$. Consequently, for simplicity and without loss of generality, we set $b_1 = 1$ and have $f_{1,t}(x) = x + b_{0,t}$. Based on the theoretical derivations presented above, the reward function suitable for RL, derived from the Lyapunov drift-plus-penalty method, is formulated as follows

$$\mathcal{R}(t) = -\frac{1}{2}\sum_{n=1}^{N}(Q_n^2(t+1) - Q_n^2(t)) - Vp(t) - b_{0,t}. \tag{30}$$

The task is simplified to determine $b_{0,t}$. Within a single time slot, $b_{0,t}$ is a constant and independent of any other quantities that may vary within the slot. Consequently, $b_{0,t}$ depends solely on fixed values within the time slot. In this study, we investigate arbitrary problems satisfying the given expression, where the only common fixed value across time slots is the queue state information, denoted as $Q(t)$. Therefore, we define $b_{0,t} = f_3(Q(t))$ as

$$\mathcal{R}(t) = -\frac{1}{2}\sum_{n=1}^{N}(Q_n^2(t+1) - Q_n^2(t)) - Vp(t) - f_3(Q(t)). \tag{31}$$

The objective is to optimize the long-term stability of the queue length. Hence, the specific time at which a particular queue length occurs is out of our concern. This leads to the following corollary.

Lemma 3. If the queue lengths $Q_n(t)$ and $Q_n(t+1)$ are swapped while keeping the penalty term $p(t)$ unchanged, the reward in Eq.(31) should remain constant.

Proof. The reward in Eq.(31) consists of queue stability part $-\frac{1}{2}\sum_{n=1}^{N}(Q_n^2(t+1) - Q_n^2(t)) - f_3(Q(t))$ and penalty part $-Vp(t)$.

We swap the queue lengths $Q_n(t)$ and $Q_n(t+1)$ while keeping the penalty term $p(t)$ unchanged:

$$\begin{aligned}Q_n^*(t) &= Q_n(t+1)\\ Q_n^*(t+1) &= Q_n(t).\end{aligned} \tag{32}$$

In a queueing system, swapping the queue lengths at two different times does not affect the queue stability. The reward in Eq.(31) should remain constant, as both the queue stability component and the penalty term remain constant.

□

From Lemma 3, we have $\mathcal{R}(t) = \mathcal{R}^*(t)$, where $R^*(t)$ is the reward when the queue lengths at the time slot t and at the time slot $t+1$ are swapped:

$$\begin{aligned}\mathcal{R}^*(t) = &-\frac{1}{2}\sum_{n=1}^{N}(Q_n^2(t) - Q_n^2(t+1)) - Vp(t)\\ &- f_3(Q(t+1)).\end{aligned} \tag{33}$$

Therefore,

$$\begin{aligned}&-\frac{1}{2}\sum_{n=1}^{N}(Q_n^2(t+1) - Q_n^2(t)) - Vp(t) - f_3(Q(t))\\ =&-\frac{1}{2}\sum_{n=1}^{N}(Q_n^2(t) - Q_n^2(t+1)) - Vp(t) - f_3(Q(t+1)).\end{aligned} \tag{34}$$

After moving the terms related to t to the right and $t+1$ to the left in Eq. (34) and simplifying, we can get

$$f_3(Q(t+1)) - \sum_{n=1}^{N} Q_n^2(t+1) = f_3(Q(t)) - \sum_{n=1}^{N} Q_n^2(t). \tag{35}$$

We can deduce that $f_3(Q(t)) - \sum_{n=1}^{N} Q_n^2(t)$ is a constant, and we set $f_3(Q(t)) - \sum_{n=1}^{N} Q_n^2(t) = b_0$. Therefore,

$$f_3(Q(t)) = \sum_{n=1}^{N} Q_n^2(t) + b_0. \tag{36}$$

B. Proposed LDPTRLQ method

The constant term in the reward function for RL is not significant. Therefore, setting $b_0 = 0$ simplifies the expression without losing any generality. Hence,

$$\begin{aligned}\mathcal{R}(t) &= -\frac{1}{2}\sum_{n=1}^{N}(Q_n^2(t+1) - Q_n^2(t)) - Vp(t) - f_3(Q(t)) \\ &= -\frac{1}{2}\sum_{n=1}^{N}(Q_n^2(t+1) - Q_n^2(t)) - Vp(t) \\ &\quad - \sum_{n=1}^{N} Q_n^2(t) - b_0 \\ &= -\frac{1}{2}\sum_{n=1}^{N}(Q_n^2(t+1) + Q_n^2(t)) - Vp(t).\end{aligned} \tag{37}$$

The reward function in Eq. (37) is the only reward function in Lemma 1 that satisfies a series of properties of RL in dealing with queueing problems. According to Theorem 3, there exists at least one form of Eq. (18) that is an optimal and static reward function. Therefore, the reward function in Eq. (37) applies to all cases.

This form differs significantly from the objective functions in the widely used $\mathcal{P}3$ or $\mathcal{P}4$ algorithms. Therefore, we demonstrate that directly using the objective functions from $\mathcal{P}3$ or $\mathcal{P}4$ in RL will result in performance degradation. Furthermore, in the context of RL, the action is defined as $\alpha \in \mathcal{A}$, while the state $s \in \mathcal{S}$ encompasses system information, including the queue state.

Additionally, when the proposed RL algorithm is reduced to performing greedy optimization within a single time slot, we have

$$\begin{aligned}&\mathcal{P}12: \min_{\alpha(t)} -\mathcal{R}(t) \\ &\text{s.t. } \alpha(t) \in \mathcal{A}(t).\end{aligned} \tag{38}$$

$\mathcal{P}12$ is equivalent to the original Lyapunov drift-plus-penalty optimization problem $\mathcal{P}3$, because $Q_n(t)$ are constant terms at time slot t. This equivalence validates the rationality of the proposed approach.

C. Adaptive Lyapunov Weight Algorithm

Although the Lyapunov weight V should typically be predetermined in most problems, predetermining it is impractical in certain scenarios, while exhaustively scanning different values of V is computationally prohibitive. Two popular approaches have been proposed to address similar issues. The first is an adaptive V based on queue length: $V(t) = c \times Q^{\beta}(t)$, where c is a pre-set coefficient, and $Q^{\beta}(t)$ is a positive correlation function with respect to queue length. The second is a water-filling-inspired approach: $V(t) = \max\{V_{\min}, \min\{V_{\max}, \kappa/(\text{target-delay} - \text{current-delay})\}$, where κ is a pre-set water-filling coefficient.

However, these methods are not applicable to the problem at hand, as RL requires V to remain quasi-static. Moreover, tuning certain parameters in these approaches remains challenging. Motivated by the essence of the problem, we propose the following algorithm.

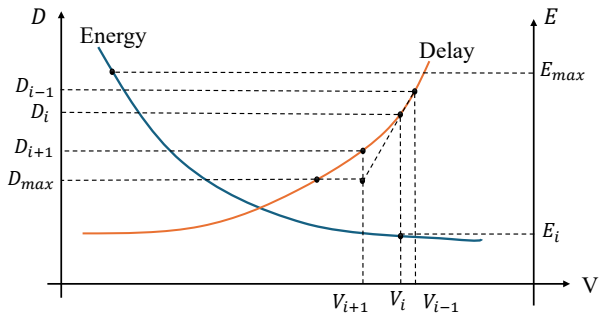


Fig. 2. Adaptive Lyapunov Weight Algorithm.

Algorithm 1 Adaptive Lyapunov Weight Method

Input: $D_{\max}$, $E_{\max}$.
Result: V.

1: Initialize a random Lyapunov weight $V_0 > 0$, the scaling factor $\eta_a = 0.1$, the maximum exploration factor $\eta_{max} = 0.5$, the minimum exploration factor $\eta_{min} = 0.01$, and $i = 0$.
2: RL is performed using V_0, and $\{D_0, E_0\}$ are obtained.
3: while $\{D_0 > D_{\max}$ and $E_0 > E_{\max}\}$ do
4: $\quad D_{\max} = (1+\eta_a)D_{\max}$. $E_{\max} = (1+\eta_a)E_{\max}$.
5: end while
6: If $D_0 > D_{\max}$ then $V_1 = (1-\eta_a)V_0$.
7: If $E_0 > E_{\max}$ then $V_1 = (1+\eta_a)V_0$.
8: repeat
9: $\quad i = i + 1$.
10: $\quad$ RL is performed using V_i, and $\{D_i, E_i\}$ are obtained.
11: $\quad$ while $\{D_i > D_{\max}$ and $E_i > E_{\max}\}$ do
12: $\quad\quad D_{\max} = (1+\eta_a)D_{\max}$. $E_{\max} = (1+\eta_a)E_{\max}$.
13: $\quad$ end while
14: $\quad$ If $D_i > D_{\max}$ then $V_{i+1} = \text{clip}(V_i + \frac{D_{\max}-D_i}{D_i - D_{i-1}}(V_i - V_{i-1}), (1-\eta_{max})V_i, (1-\eta_{min})V_i)$.
15: $\quad$ If $E_i > E_{\max}$ then $V_{i+1} = \text{clip}(V_i + \frac{E_{\max}-E_i}{E_i - E_{i-1}}(V_i - V_{i-1}), (1+\eta_{min})V_i, (1+\eta_{max})V_i)$.
16: until $\{D_i \le D_{\max}$ and $E_i \le E_{\max}\}$ or upper limit of iterations.

We assume that the overall objective is to satisfy the maximum delay constraint $D_{\max}$ and the maximum energy consumption constraint $E_{\max}$. The delay D is related to the queue length Q, while the energy consumption E is associated with the Lyapunov penalty term p. Specifically, the delay D increases with larger values of V, whereas the energy consumption E decreases as V increases. To find an appropriate V that balances these trade-offs, we employ the secant method illustrated in Fig. 2 and Algorithm 1.

To ensure robust convergence and prevent system instability during the nascent exploration phase, we implement a structured initialization and threshold relaxation strategy. Since the secant method requires two distinct initial points to estimate the local gradient, the algorithm begins by stochastically generating an initial weight V_0. Subsequently, a second point V_1 is derived by applying a

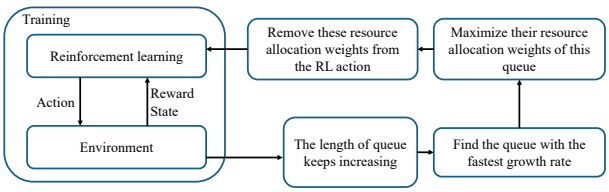


Fig. 3. Saturated Queueing Algorithm.

fixed exploration step size to V_0.

If the initial system performance $\{D_i, E_i\}$ violates both the delay constraint $D_{\max}$ and the energy constraint $E_{\max}$ simultaneously, an adaptive scaling mechanism is triggered to relax these thresholds by a factor η_a until at least one constraint becomes feasible, thereby establishing a valid starting region for the iterative process.

In scenarios where the delay constraint $D_{\max}$ is violated, the update rule for V_{i+1} is formulated based on the geometric principles of the Secant method. By assuming that the points (D_{i-1}, V_{i-1}), (D_i, V_i), and $(D_{\max}, V_{i+1})$ are collinear as illustrated in Fig. 2, the next iteration value is derived through linear interpolation as follows:

$$\begin{aligned} V_{i+1} =& \text{clip}(V_i + \frac{D_{\max} - D_i}{D_i - D_{i-1}}(V_i - V_{i-1}), \\ & (1-\eta_{\max})V_i, (1-\eta_{\min})V_i), \end{aligned} \tag{39}$$

where the function $\text{clip}(x, a, b) = \max(a, \min(x, b))$ restricts the variable to a predefined interval $[a, b]$.

The introduction of the clip function is critical. It prevents excessive fluctuations in V_i that could lead to algorithmic divergence. Furthermore, by enforcing a minimum exploration bound $(1-\eta_{\min})V_i$, we ensure that the algorithm does not prematurely stagnate, thereby driving the delay D_i to converge toward or below the target $D_{\max}$ rather than merely approaching it asymptotically.

A symmetrical update logic is applied when the energy consumption constraint $E_{\max}$ is violated.

D. Saturated Queueing Algorithm

RL training requires an effective reward function. However, when the arrival rate approaches the service rate, the queue length can easily grow unbounded. Under such conditions, unbounded queue growth renders the reward function partially ineffective, as the penalties or rewards lose their discriminative power. Consequently, in scenarios where certain queues operate near saturation, the RL policy tends to oscillate among invalid solutions, significantly hindering algorithm convergence. To address this challenge when the arrival rate approaches the service capacity limit, we design a dedicated bailout mechanism as shown in Fig. 3.

This algorithm is triggered when the RL training fails to converge due to unbounded growth in certain queues. It first identifies the queue exhibiting the fastest growth rate, then sets the resource allocation coefficient for that queue to its maximum value, and subsequently removes this coefficient from the action space of the RL agent. The objective of this mechanism is to detect saturated queues and directly maximize their resource allocation, thereby preventing unbounded queue growth and stabilizing the system.

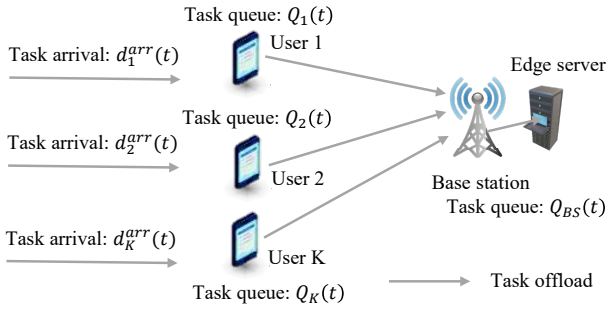


Fig. 4. System model of mobile edge computing.

IV. Numerical Results and Analysis

We introduce two queueing-related practical problems to evaluate the performance of our proposed algorithm.

A. Mobile Edge Computing

We consider a system model for mobile edge computing, comprising K mobile users and a single base station as shown in Fig. 4. The base station is equipped with an edge computing server dedicated to processing computational tasks. Each mobile user generates computational tasks in each time slot with a certain probability, where task arrivals follow a Poisson process with an arrival rate of λ. The size of each task is uniformly distributed over the interval $(0, d_{\max})$. For each mobile user, a task queue $Q_k(t)$ is maintained, with a local computing speed (bits/s) denoted by $c_k^L(t)$ and an upper bound on the computing speed (bits/s) given by $c_{\max,k}^L(t)$.

The evolution of the queue lengths at the kth mobile user at time slot t can be described by the following queueing model:

$$Q_k(t+1) = \max\{Q_k(t) - d_k^{\text{dep}}(t), 0\} + d_k^{\text{arr}}(t), \forall k, \tag{40}$$

where $d_k^{\text{arr}}(t)$ is the total length of the newly generated task demand at time slot t, and $d_k^{\text{dep}}(t)$ represents the number of tasks departing from the task queue at time slot t, which comprises the tasks processed locally within the time slot t and those offloaded to the edge computing server:

$$d_k^{\text{dep}}(t) = R_k(t)\Delta T + c_k^L(t)\Delta T, \forall k, \tag{41}$$

where ΔT is the length of each time slot, and $R_k(t)$ is the offloading data rate (bit/s). Task offloading is facilitated through wireless communication. We adopt a fundamental wireless communication model wherein each user is assigned an independent orthogonal channel, each with a bandwidth of B. The transmission rate from user k to the base station, which corresponds to the task offloading rate, is given by

$$R_k(t) = B\log_2\left(1 + \frac{\omega_k^{\text{WL}}(t)P_k(t)}{\sigma}\right), \forall k, \tag{42}$$

where $\omega_k^{\mathrm{WL}}(t)$ denotes the channel gain at time slot t, $P_k(t)$ represents the transmission power from user k to the base station at time slot t, and σ is the channel noise power. The evolution of the task queue lengths at the edge server at the base station at time slot t can be described by the following queueing model

$$Q_{\mathrm{BS}}(t+1) = \max\{Q_{\mathrm{BS}}(t) - d_{\mathrm{BS}}^{\mathrm{dep}}(t), 0\} + d_{\mathrm{BS}}^{\mathrm{arr}}(t), \quad (43)$$

where $d_{\mathrm{BS}}^{\mathrm{arr}}(t)$ is the total length of the newly received task demand at the base station at time slot t: $d_{\mathrm{BS}}^{\mathrm{arr}}(t) = \sum_{k=1}^{K} R_k(t)\Delta T$, and $d_{\mathrm{BS}}^{\mathrm{dep}}(t)$ represents the number of tasks departing from the task queue $Q_{\mathrm{BS}}(t)$ at time slot t, which comprises the tasks processed within the time slot t at the edge server: $d_{\mathrm{BS}}^{\mathrm{dep}}(t) = c_{\mathrm{BS}}^{E}(t)\Delta T$. $c_{\mathrm{BS}}^{E}(t)$ is the task processing speed (bit/s) at the edge server at the base station at time slot t with an upper bound given by $c_{\max,\mathrm{BS}}^{E}(t)$. The energy consumption within each time slot comprises both the energy expended on communication and the energy expended on computation:

$$\begin{aligned} E^{\mathbb{A}}(t) =& E^{\mathbb{W}C}(t) + E^{\mathbb{C}\mathbb{C}}(t) \\ =& \sum_{k=1}^{K} P_k(t)\Delta T + \sum_{k=1}^{K} \eta_k c_k^{L}(t)\Delta T + \eta_{\mathrm{BS}} c_{\mathrm{BS}}^{E}(t)\Delta T, \end{aligned} \quad (44)$$

where η_k is the computing energy consumption per bit when the task is processed at the kth mobile user, and η_{BS} is the computing energy consumption per bit when the task is processed at the edge server at the base station.

The reward function at time slot t of the proposed LDPTRLQ algorithm, as derived in Eq. (37) of Section III, is applied to this specific edge computing problem, resulting in the following formula

$$\begin{aligned} \mathcal{R}(t) =& -\frac{1}{2}\sum_{k=1}^{N}(Q_k^2(t+1) + Q_k^2(t)) \\ & -\frac{1}{2}(Q_{\mathrm{BS}}^2(t+1) + Q_{\mathrm{BS}}^2(t)) - VE^{\mathbb{A}}(t). \end{aligned} \quad (45)$$

The action at time slot t contains the local computing speed of the mobile user, the computing speed of the edge computing server, and the power allocation for wireless communication:

$$\boldsymbol{a}(t) = \{c_k^{L}(t), c_{\mathrm{BS}}^{E}(t), P_k(t), \forall k\}. \quad (46)$$

To ensure the physical feasibility of decisions and maintain training stability, we incorporate a projection layer into the output of the actor network. In the considered MEC scenario, an action is deemed invalid if the sum of the offloaded task volume and the locally processed volume exceeds the total available workload in the task buffer. The action actually used is the closest legal action. The state at time slot t comprises the task queue length at the mobile user, the task queue length at the edge computing server, and the wireless channel gain:

$$\boldsymbol{s}(t) = \{Q_k(t), Q_{\mathrm{BS}}(t), \omega_k^{\mathrm{WL}}(t), \forall k\}. \quad (47)$$

Our simulation system model consists of 1 base station with 1 edge computing server, serving 10 mobile users.

TABLE II
Simulation settings

Parameter	Value	Parameter	Value
Bandwidth	10kHz	Noise	3.16×10^{-11}
Arrival model	Poisson	Channel Model	Rayleigh
$c_{\max,k}^{L}$	1000	$c_{\max,\mathrm{BS}}^{E}$	5000
λ	2	K	10
Activ. func.	ReLU	PPO agent	5 layers
Layer	64 neurons	Mini batch	128
Reply memory	10000	Discount factor	0.95
Steps per episode	500	Maximum episode	1000

The type of RL agent is proximal policy optimization (PPO). The PPO agent has 5 fully connected layers with 64 neurons per layer equipped with the ReLU activation function. We set the PPO clipping parameter to 0.2. The discount factor is 0.95. The maximum number of steps per episode is 500, and the maximum number of episodes is 1000. The simulation environment is equipped with an Intel® Core™ i9-13900HX CPU and an RTX 4080 GPU. The simulation settings are also shown in TABLE II.

To verify the efficiency of our proposed algorithm, we performed a comparative analysis against several different algorithms. These comparisons aim to highlight the performance enhancements achieved through our method under various standard scenarios used in the field:

- Original LDPRLQ: This is the original Lyapunov Drift Plus Penalty with RL (LDPRLQ) algorithm, as utilized in several academic papers [24]. It directly uses the negative of the Lyapunov drift plus penalty function as the reward for RL.
- Simplified LDPRLQ: A variant adopted in [25], where the Lyapunov drift plus penalty function is approximated as follows

$$\mathcal{C}(t) = \sum_{n=1}^{N} Q_n(t)(Q_n(t+1) - Q_n(t)) + Vp(t). \quad (48)$$

  The negative of $\mathcal{C}(t)$ is used as the reward for RL.
- LERL: Latency-energy tradeoff with RL. An algorithm unrelated to the Lyapunov drift plus penalty framework, which optimizes a weighted sum of latency and energy consumption, as presented in [33]. In the context of our problem, latency corresponds to the queue length, thus directly optimizing the weighted sum of queue length and the penalty term.
- PPO-Lagrangian: Proximal Policy Optimization Lagrangian is a widely recognized baseline in constrained RL. It extends the standard PPO algorithm to handle CMDP by incorporating the Lagrangian multiplier method. The objective function is augmented with a penalty term, where the Lagrange multiplie scales the cost surrogate based on the degree of constraint violation.
- LDPTRLQ(DQN): A modification of our proposed algorithm where the default PPO agent is replaced with a Deep Q-Network (DQN) agent.

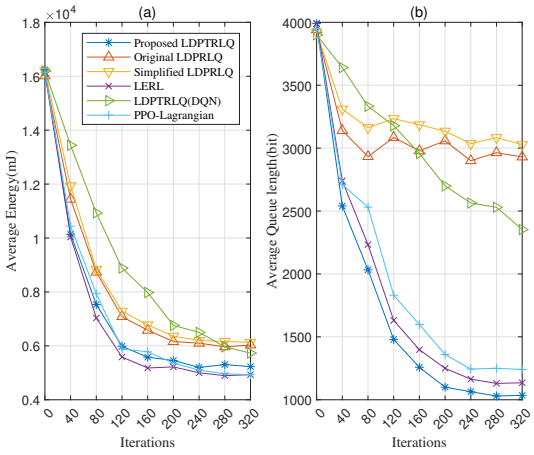


Fig. 5. The convergence behavior of our proposed LDPTRLQ with benchmarks.

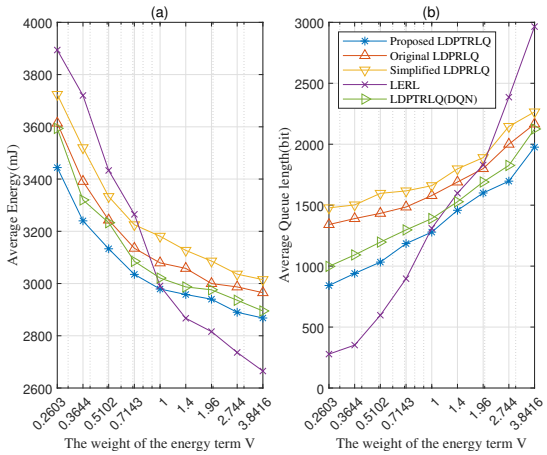


Fig. 6. The energy and queue length behavior of our proposed LDPTRLQ with benchmarks under varying weights.

1) Convergence speed comparison: Fig. 5 illustrates the convergence behavior of the proposed LDPTRLQ algorithm alongside four baseline methods. Since the trade-off coefficient in the LERL algorithm differs in meaning from the Lyapunov weight V in our proposed approach, a preliminary traversal of LERL's trade-off coefficient was conducted. This enabled the selection of a coefficient value that aligns the energy consumption and queue length metrics with those of our algorithm, facilitating a fair comparison under consistent conditions. Simulation results reveal that the proposed LDPTRLQ algorithm achieves a balanced trade-off, significantly reducing both average energy consumption and average queue length. Its convergence speed surpasses that of Original LDPRLQ, Simplified LDPRLQ, and LDPTRLQ(DQN), demonstrating enhanced optimization efficiency. While the LERL algorithm yields results comparable to our method, it requires careful tuning of the trade-off coefficient to ensure queue stability, whereas our algorithm inherently maintains stability without such adjustments. Although PPO-Lagrangian exhibits competitive performance comparable to LERL and the proposed LDPRLQ in terms of energy and latency, its efficacy heavily relies on the manual pre-specification of constraint violation penalties for different scenarios. Furthermore, the slower convergence of LDPTRLQ(DQN) compared to the proposed LDPTRLQ suggests that, under these problem conditions, the PPO agent outperforms the DQN agent. The superior convergence speed of our LDPTRLQ over Original LDPRLQ and Simplified LDPRLQ corroborates our theoretical derivations, underscoring deficiencies in these baseline methods that our approach effectively mitigates.

2) Impact of different Lyapunov weight of proposed LDPTRLQ and benchmarks: Fig. 6 presents the performance of the proposed LDPTRLQ algorithm alongside four baseline methods under varying Lyapunov weight conditions. Given that the trade-off weight in the LERL algorithm differs in interpretation from the Lyapunov weight in our approach, a standardization of the weight factors

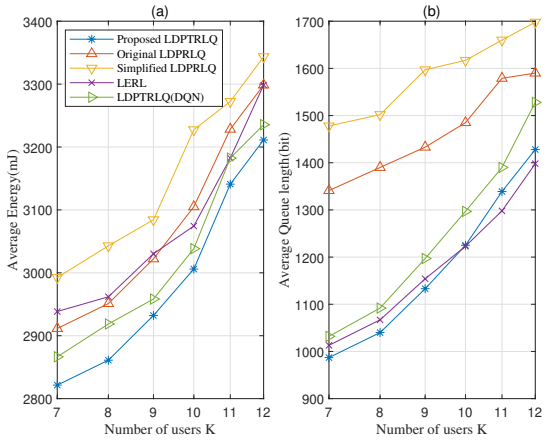


Fig. 7. The energy and queue length behavior of our proposed LDPTRLQ with benchmarks under varying number of users.

was performed in advance. Specifically, a coarse traversal of the trade-off coefficient in LERL was conducted, and the weight yielding energy consumption and queue length values closest to those of the proposed LDPTRLQ algorithm was normalized to 1. This normalization enables a consistent comparison across the algorithms. Simulation results demonstrate that the proposed LDPTRLQ algorithm outperforms Original LDPRLQ and Simplified LDPRLQ in terms of both average queue length and average energy consumption across all tested conditions. This improvement validates our theoretical findings, confirming that the proposed algorithm enhances the Lyapunov Drift-Plus-Penalty framework for RL applications. In contrast, LDPTRLQ(DQN) consistently exhibits inferior performance relative to the proposed LDPTRLQ, indicating that the PPO agent is more effective than the DQN agent in this application context. While the LERL algorithm achieves performance comparable to that of LDPTRLQ, it exhibits greater sensitivity to weight variations, making it challenging to identify a weight that ensures queue stability.

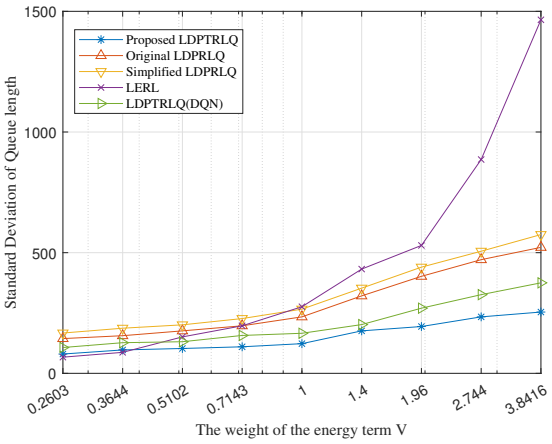


Fig. 8. The standard deviation queue length behavior of our proposed LDPTRLQ with benchmarks under varying weights.

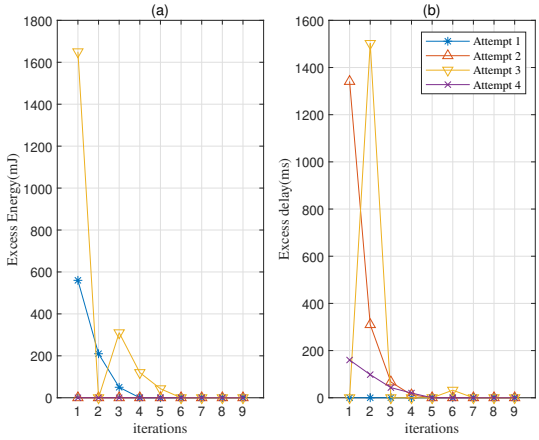


Fig. 9. The convergence behavior of our proposed adaptive Lyapunov weight algorithm.

3) Impact of different number of users of proposed LDPTRLQ and benchmarks: Fig. 7 presents the performance of the proposed LDPTRLQ algorithm alongside four baseline methods under varying number of users. The simulation results demonstrate that the proposed LDPTRLQ algorithm consistently outperforms the original LDPRLQ and the simplified LDPRLQ variants across all test configurations in terms of both average queue length and average energy consumption. This superiority highlights the efficacy of integrating trust region constraints with the Lyapunov-based framework. Specifically, the performance of LDPTRLQ(DQN) is observed to be systematically inferior to the proposed PPO-based LDPTRLQ. While the LERL algorithm achieves competitive results in certain scenarios, its performance is marginally lower than that of LDPRLQ. Furthermore, as the number of users increases, the average energy consumption and queue length for all algorithms exhibit an upward trend. However, the growth rate remains within a reasonable and manageable range, which validates the scalability of the proposed framework in handling high-dimensional state and action spaces.

4) The queue length standard deviation of proposed LDPTRLQ and benchmarks: Fig. 8 illustrates the queue length standard deviation of the proposed LDPTRLQ algorithm and four baseline methods under varying Lyapunov weight conditions. The same weight factor standardization approach as described previously was applied. The results indicate that the proposed LDPTRLQ algorithm achieves the smallest queue length standard deviation in most scenarios, with LERL exhibiting a marginally lower standard deviation only when the queue weight factor is significantly elevated. This suggests that our proposed algorithm ensures superior queue stability compared to all baseline methods. The enhanced stability of LDPTRLQ stems from its incorporation of the queue length as a squared term in the RL reward function, in contrast to LERL, which uses a linear term. Consequently, LERL is less sensitive to occasional excessive queue lengths, functioning primarily as a trade-off mechanism between queue length and energy consumption rather than an optimization strategy prioritizing queue stability. Similarly, the proposed LDPTRLQ algorithm consistently outperforms Original LDPRLQ and Simplified LDPRLQ in terms of queue length standard deviation, demonstrating an improvement over conventional approaches that integrate Lyapunov Drift-Plus-Penalty with RL. Furthermore, the superior performance of LDPTRLQ over LDPTRLQ(DQN) reinforces the finding that, in this application context, the PPO agent outperforms the DQN agent.

5) Convergence behavior of the adaptive Lyapunov weight algorithm: Fig. 9 illustrates the convergence performance of our proposed dynamic Lyapunov weight algorithm across four independent runs. The convergence speed is influenced by the initial random value of the Lyapunov weight V. In all cases, the algorithm rapidly converges to a solution where neither the energy consumption exceeds the prescribed maximum $E_{\max}$ nor the delay surpasses the maximum $D_{\max}$. This demonstrates the effectiveness of the proposed dynamic Lyapunov weight adjustment algorithm in efficiently identifying an appropriate weight V.

6) Convergence behavior of the saturated queueing algorithm: Fig. 10 illustrates the convergence performance of the proposed saturated queue bailout mechanism. In the simulation, one user is configured with a high task arrival rate approaching its computational capacity limit. The results show that, without the saturated queue mechanism, the RL algorithm experiences considerable difficulty in converging. When the policy explores actions that lead to unbounded queue accumulation, the reward function partially loses its discriminative power, thereby hindering effective policy improvement. In contrast, with the proposed saturated queue bailout mechanism enabled, the algorithm promptly identifies the saturated user and directly sets its resource allocation coefficient to the maximum value. This intervention effectively prevents un-

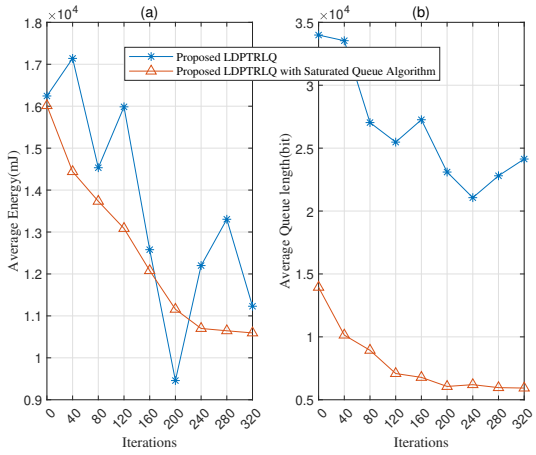


Fig. 10. The convergence behavior of our proposed saturated queueing algorithm.

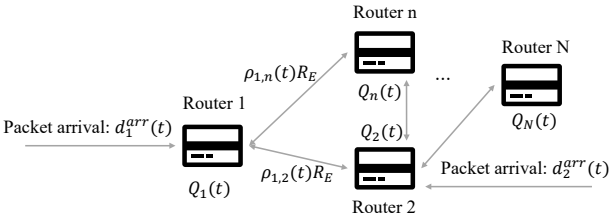


Fig. 11. System model of network routing.

bounded queue growth and enables the overall algorithm to achieve stable and proper convergence.

B. Network Routing

In our simulation, we developed a method for constructing a randomly generated network topology for analysis as shown in Fig. 11. The generation process begins with the creation of 20 nodes, each assigned a random two-dimensional coordinate (x, y). The values for x and y are uniformly distributed between 0 and 100, ensuring that the node distribution covers the entire designated area evenly. Each node is programmed to potentially connect to its 5 nearest nodes. The probability of each of these connections being established is set at 50%, introducing a stochastic element to the network structure that mimics real-world irregularities and network dynamics. If the topology graphs are separated, we connect the closest nodes between them. The achievable data rate with each connection, denoted as R, is calculated using the formula $R = \rho R_E$, where $\rho \in [0.3, 1.7]$ represents a scaling factor and R_E is the average data rate between the nodes. The uniformly distributed scaling factor ρ allows the connection speed to reflect a range of potential network conditions, from favorable to adverse.

In [10], the Lyapunov drift theory (with $V = 0$, i.e., without any penalty term) was employed to mathematically analyze the conventional backpressure routing algorithm. These studies demonstrated that the backpressure routing algorithm is consistent with the Lyapunov drift framework. Specifically, under the condition $V = 0$, we can rewrite $\mathcal{P}4$ as

$$\begin{aligned} &\mathcal{P}12: \min_{\alpha(t)} \sum_{n=1}^{N} Q_n(t)(d_n^{\text{arr}}(t) - d_n^{\text{dep}}(t)) \\ &\text{s.t. } \alpha(t) \in \mathcal{A}(t), \end{aligned} \tag{49}$$

where $\alpha(t)$ denotes the routing action. These patterns are designed such that each queue has exactly one next-hop routing target. Problem $\mathcal{P}12$ is formulated as a linear programming problem, which can be solved by the CVX optimization toolbox. This solution approach is equivalent to the backpressure routing algorithm. Similarly, our proposed LDPTRLQ algorithm is capable of addressing this routing problem.

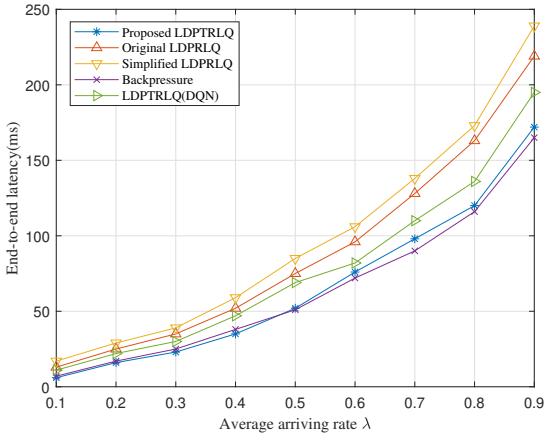


Fig. 12. The end-to-end latency versus the average arriving rate of our proposed LDPTRLQ and benchmarks.

1) Impact of different arriving rate of proposed LDPTRLQ and benchmarks: The performance of our proposed LDPTRLQ algorithm, as illustrated in Fig. 12, closely approximates that of the conventional backpressure routing algorithm. Notably, under low network congestion, LDPTRLQ exhibits a slight performance advantage over the backpressure algorithm. However, despite this advantage under specific conditions, the computational resource consumption of our proposed algorithm significantly exceeds that of the backpressure algorithm, which requires minimal computational resources. Consequently, in the absence of a penalty term, the LDPTRLQ algorithm does not outperform the backpressure algorithm because the problem reduces to a pure queue-stability task for which the backpressure algorithm is already known to provide a near-optimal solution with low computational overhead. Furthermore, the higher latency observed with LDPTRLQ(DQN) suggests that the PPO agent outperforms the DQN agent in this environment. The numerical results also demonstrate that our proposed LDPTRLQ algorithm exceeds those of other algorithms combining Lyapunov drift-plus-penalty with RL.

2) Impact of different Lyapunov weight of proposed LDPTRLQ and benchmarks: We introduce the energy model $E(t) = \eta_2 R^2(t) + \eta_1 R(t)$, where η_1 and η_2 are

TABLE III
Algorithm comparison

Algorithm	Guarantee of Queue stability	Trade-off between two objectives	At least one metric is optimal
Original LDPRLQ	✓	✓	×
Simplified LDPRLQ	✓	✓	×
LERL	×	✓	✓
PPO-Lagrangian	×	✓	✓
LDPTRLQ(DQN)	✓	✓	×
Backpressure	✓	×	✓
Proposed LDPTRLQ	✓	✓	✓

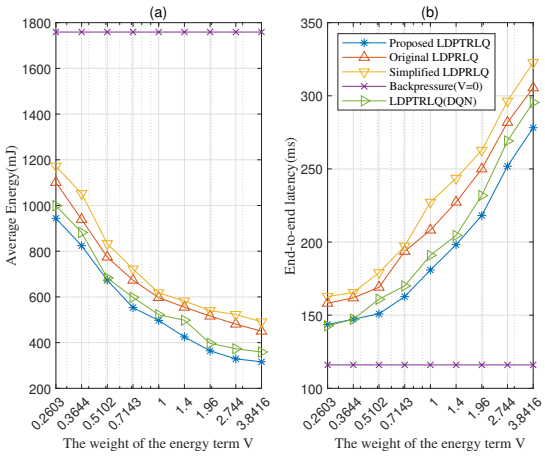


Fig. 13. The end-to-end latency versus the weight of energy term V our proposed LDPTRLQ and benchmarks.

known coefficients, and $R(t)$ is the transmission data rate at time slot t. Furthermore, we subsequently address the comprehensive Lyapunov drift-plus-penalty optimization framework. Incorporating complex penalty terms into the backpressure routing algorithm is challenging due to the non-convex nature of the optimization problem, which renders it difficult to solve. Consequently, for the backpressure routing algorithm, we fix the trade-off parameter $V = 0$. As illustrated in Fig. 13, simulation results demonstrate that the proposed LDPTRLQ algorithm outperforms all other algorithms except the backpressure routing algorithm, highlighting its superiority over existing approaches combining Lyapunov drift-plus-penalty with RL. Moreover, while accounting for energy consumption, our algorithm achieves latency performance comparable to that of the backpressure algorithm to a certain extent. This ability to incorporate complex optimization factors while maintaining effective routing performance represents a key advantage of the proposed LDPTRLQ algorithm over the backpressure routing algorithm.

TABLE III summarizes the strengths of our proposed algorithm and various benchmarks.

V. Conclusion

In this paper, we proposed LDPTRLQ, an enhanced algorithm integrating the Lyapunov Drift-Plus-Penalty framework with RL to address the optimization challenges posed by emerging complicated systems. Through theoretical derivations, we established an effective approach that outperforms conventional methods by effectively combining the short-term greediness of Lyapunov Drift-Plus-Penalty with the long-term optimization capabilities of RL. Simulation results in the MEC environment demonstrate that LDPTRLQ consistently surpasses baseline algorithms. These findings validate our theoretical analysis, highlighting the algorithm's robustness and efficiency. While LERL exhibits competitive performance, its sensitivity to weight tuning underscores LDPTRLQ's practical advantage in maintaining queue stability without extensive parameter adjustments. Simulation results in the routing environment demonstrate that, compared to the conventional backpressure routing algorithm, the proposed LDPTRLQ algorithm effectively accommodates a wide range of complex optimization objectives. Furthermore, it outperforms other algorithms that integrate Lyapunov drift-plus-penalty with RL. Future work will focus on validating these results in real-world IoT deployments and exploring adaptive weight mechanisms to further enhance performance under dynamic conditions.

References

[1] J. Núñez-Martínez and J. Mangues-Bafalluy, "Distributed Lyapunov drift-plus-penalty routing for wifi mesh networks with adaptive penalty weight," in 2012 IEEE International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM). IEEE, 2012, pp. 1–6.

[2] N. Li, Y. Hu, Y. Chen, and B. Zeng, "Lyapunov optimized resource management for multiuser mobile video streaming," IEEE Trans. Circuits Systems Video Technol., vol. 29, no. 6, pp. 1795–1805, 2018.

[3] T. Shuminoski and T. Janevski, "Lyapunov optimization framework for 5G mobile nodes with multi-homing," IEEE Commun. Letters, vol. 20, no. 5, pp. 1026–1029, 2016.

[4] H. Yu and M. J. Neely, "On the convergence time of the drift-plus-penalty algorithm for strongly convex programs," in 2015 54th IEEE Conference on Decision and Control (CDC). IEEE, 2015, pp. 2673–2679.

[5] M. Hadi, M. R. Pakravan, and E. Agrell, "Dynamic resource allocation in metro elastic optical networks using Lyapunov drift optimization," J. Optical Commun. Network., vol. 11, no. 6, pp. 250–259, 2019.

[6] M. Neely, Stochastic network optimization with application to communication and queueing systems. Springer Nature, 2022.

[7] S. Kerimov, I. Ashlagi, and I. Gurvich, "On the optimality of greedy policies in dynamic matching," Operations Research, vol. 73, no. 1, pp. 560–582, 2025.

[8] J. Yu, X. Liu, Y. Gao, C. Zhang, and W. Zhang, "Deep learning for channel tracking in IRS-assisted UAV communication systems," IEEE Trans. Wireless Commun., vol. 21, no. 9, pp. 7711–7722, 2022.

[9] X. Huang, S. Leng, S. Maharjan, and Y. Zhang, "Multi-agent deep reinforcement learning for computation offloading and interference coordination in small cell networks," IEEE Trans. Vehicular Technol., vol. 70, no. 9, pp. 9282–9293, 2021.

[10] M. Neely, Stochastic network optimization with application to communication and queueing systems. Morgan & Claypool Publishers, 2010.

[11] D. U. Kim, K. Kim, S. M. Kang, S.-B. Park, and C. S. Hong, "When Lyapunov drift meets DRL: Energy efficient resource allocation for IoT data collecting," in 2023 International Conference on Advanced Technologies for Communications (ATC). IEEE, 2023, pp. 382–387.

[12] C. Xu, P. Zhang, and H. Yu, "Lyapunov-guided resource allocation and task scheduling for edge computing cognitive radio networks via deep reinforcement learning," IEEE Sensors J., vol. 25, no. 7, pp. 12 253–12 264, 2025.

[13] L. Bracciale and P. Loreti, "Lyapunov drift-plus-penalty optimization for queues with finite capacity," IEEE Commun. Lett., vol. 24, no. 11, pp. 2555–2558, 2020.

[14] H. Jiang, X. Dai, Z. Xiao, and A. Iyengar, "Joint task offloading and resource allocation for energy-constrained mobile edge computing," IEEE Trans. Mobile Computing, vol. 22, no. 7, pp. 4000–4015, 2022.

[15] C.-H. Hu, Z. Chen, and E. G. Larsson, "Energy-efficient federated edge learning with streaming data: A lyapunov optimization approach," IEEE Trans. Commun., vol. 73, no. 2, pp. 1142–1156, 2024.

[16] Q. Meng, S. Hussain, F. Luo, Z. Wang, and X. Jin, "An online reinforcement learning-based energy management strategy for microgrids with centralized control," IEEE Trans. Industry Applications, vol. 61, no. 1, pp. 1501–1510, 2024.

[17] H. Wang, Z. Liu, G. Hu, X. Wang, and Z. Han, "Offline meta-reinforcement learning for active pantograph control in high-speed railways," IEEE Trans. Industrial Informatics, vol. 20, no. 8, pp. 10 669–10 679, 2024.

[18] J. Jiang and Y. Ye, "Achieving instance-dependent sample complexity for constrained markov decision process," arXiv preprint arXiv:2402.16324, 2024.

[19] Y. Efroni, S. Mannor, and M. Pirotta, "Exploration-exploitation in constrained MDPs," arXiv preprint arXiv:2003.02189, 2020.

[20] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in Proceedings of the 34th International Conference on Machine Learning, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 06–11 Aug 2017, pp. 22–31.

[21] Y. Cao, H. Zhao, Y. Cheng, T. Shu, Y. Chen, G. Liu, G. Liang, J. Zhao, J. Yan, and Y. Li, "Survey on large language model-enhanced reinforcement learning: Concept, taxonomy, and methods," IEEE Trans. Neural Networks Learning Systems, vol. 36, no. 6, pp. 9737–9757, 2024.

[22] J. Valente, J. António, C. Mora, and S. Jardim, "Developments in image processing using deep learning and reinforcement learning," Journal of Imaging, vol. 9, no. 10, p. 207, 2023.

[23] T. Nakayashiki and T. Kaneko, "Maximum entropy reinforcement learning in two-player perfect information games," in 2021 IEEE Symposium Series on Computational Intelligence (SSCI). IEEE, 2021, pp. 01–08.

[24] Y. Dai, K. Zhang, S. Maharjan, and Y. Zhang, "Deep reinforcement learning for stochastic computation offloading in digital twin networks," IEEE Trans. Industrial Informatics, vol. 17, no. 7, pp. 4968–4977, 2020.

[25] S. Bi, L. Huang, H. Wang, and Y. J. A. Zhang, "Lyapunov-guided deep reinforcement learning for stable online computation offloading in mobile-edge computing networks," IEEE Trans. Wireless Commun., vol. 20, no. 11, pp. 7519–7537, 2021.

[26] H. Liao, Z. Zhou, X. Zhao, L. Zhang, S. Mumtaz, A. Jolfaei, S. H. Ahmed, and A. K. Bashir, "Learning-based context-aware resource allocation for edge-computing-empowered industrial IoT," IEEE Internet Things J., vol. 7, no. 5, pp. 4260–4277, 2019.

[27] Y. Zhou, L. Feng, X. Jiang, W. Li, and F. Zhou, "Predictable wireless networked scheduling for bridging hybrid time-sensitive and real-time services," IEEE Trans. Commun., vol. 72, no. 6, pp. 3664–3680, 2024.

[28] Y. Jia, C. Zhang, Y. Huang, and W. Zhang, "Lyapunov optimization based mobile edge computing for internet of vehicles systems," IEEE Trans. commun., vol. 70, no. 11, pp. 7418–7433, 2022.

[29] X. Dai, Z. Xiao, H. Jiang, and J. C. Lui, "UAV-assisted task offloading in vehicular edge computing networks," IEEE Trans. Mobile Computing, vol. 23, no. 4, pp. 2520–2534, 2023.

[30] G. Sun, Z. Wang, H. Su, H. Yu, B. Lei, and M. Guizani, "Profit maximization of independent task offloading in MEC-enabled 5G internet of vehicles," IEEE Trans. Intelligent Transportat. Systems, vol. 25, no. 11, pp. 16 449–16 461, 2024.

[31] W. Xu, Y. Yuan, and D. H. K. Tsang, "Enhancing fog computing through intelligent reflecting surface assistance: A Lyapunov driven reinforcement learning approach," in 2024 IEEE 10th World Forum on Internet of Things (WF-IoT). IEEE, 2024, pp. 759–764.

[32] W. Xu, C. Pan, Y. Yuan, Y. Wu, and D. H. K. Tsang, "Reconfigurable intelligent surface aided mobile fog computing: A space aggregation-based lyapunov driven reinforcement learning approach," IEEE Trans. Mobile Computing, vol. 25, no. 3, pp. 3809–3823, 2026.

[33] W. Xu, J. Yu, Y. Wu, and D. H. K. Tsang, "Energy-latency aware intelligent reflecting surface aided multi-cell mobile edge computing," IEEE Trans. Green Commun. Network., vol. 8, no. 1, pp. 362–374, 2023.